

Theory Behind GPUSolver::smallestCondensedMechanicalEigenpair

1 Purpose

The function `GPUSolver::smallestCondensedMechanicalEigenpair` computes the smallest eigenvalue and corresponding eigenvector of the mechanical stability problem after eliminating the electric potential degrees of freedom. In the flexoelectric problem, the unknown vector is ordered as

$$\mathbf{z} = \begin{bmatrix} \mathbf{u} \\ \phi \end{bmatrix},$$

where $\mathbf{u}$ contains the displacement degrees of freedom and ϕ contains the electric potential degrees of freedom.

At a converged nonlinear state, the assembled tangent/Hessian matrix has the block form

$$\mathbf{H} = \begin{bmatrix} \mathbf{H}_{uu} & \mathbf{H}_{u\phi} \\ \mathbf{H}_{\phi u} & \mathbf{H}_{\phi\phi} \end{bmatrix}.$$

The goal is to test mechanical stability while allowing the electric potential field to relax consistently with the mechanical perturbation.

2 Condensation of the Electric Potential

For a perturbation $(\delta\mathbf{u}, \delta\phi)$, the quadratic second variation is

$$\delta^2\Pi = \begin{bmatrix} \delta\mathbf{u} \\ \delta\phi \end{bmatrix}^T \begin{bmatrix} \mathbf{H}_{uu} & \mathbf{H}_{u\phi} \\ \mathbf{H}_{\phi u} & \mathbf{H}_{\phi\phi} \end{bmatrix} \begin{bmatrix} \delta\mathbf{u} \\ \delta\phi \end{bmatrix}.$$

For a given mechanical perturbation $\delta\mathbf{u}$, the compatible electric perturbation is obtained by enforcing stationarity with respect to $\delta\phi$:

$$\mathbf{H}_{\phi u}\delta\mathbf{u} + \mathbf{H}_{\phi\phi}\delta\phi = \mathbf{0}.$$

Assuming $\mathbf{H}_{\phi\phi}$ is nonsingular,

$$\delta\phi = -\mathbf{H}_{\phi\phi}^{-1}\mathbf{H}_{\phi u}\delta\mathbf{u}.$$

Substituting this expression into the quadratic form gives the condensed mechanical Hessian

$$\widehat{\mathbf{H}} = \mathbf{H}_{uu} - \mathbf{H}_{u\phi}\mathbf{H}_{\phi\phi}^{-1}\mathbf{H}_{\phi u}.$$

Therefore the condensed mechanical eigenvalue problem is

$$\widehat{\mathbf{H}}\mathbf{x} = \lambda\mathbf{x}.$$

The smallest eigenvalue $\lambda_{\min}$ is used as the stability indicator. When it crosses zero, the current equilibrium loses stability in the condensed mechanical sense.

3 Computing the Smallest Eigenvector by Inverse Iteration

The purpose of this section is to explain how the code updates the mechanical eigenvector during the outer iteration. At this stage, assume the code already has a normalized mechanical vector

$$\mathbf{x}_k.$$

The goal of one inverse-iteration step is to compute

$$\mathbf{y}_k = \widehat{\mathbf{H}}^{-1} \mathbf{x}_k.$$

If this operation can be performed repeatedly, then the direction of $\mathbf{x}_k$ will converge to the eigenvector associated with the eigenvalue of $\widehat{\mathbf{H}}$ closest to zero.

Applying $\widehat{\mathbf{H}}^{-1}$ Without Forming It

The code does not explicitly form $\widehat{\mathbf{H}}^{-1}$. It also does not need to explicitly form a dense inverse of the Schur complement. Instead, it applies $\widehat{\mathbf{H}}^{-1}$ to the current vector $\mathbf{x}_k$ by solving one full coupled linear system.

The right-hand side of this coupled solve is

$$\begin{bmatrix} \mathbf{x}_k \\ \mathbf{0} \end{bmatrix}.$$

The upper part is the current mechanical vector, and the lower part is zero because we are applying a purely mechanical load in the condensed mechanical problem.

The code solves

$$\begin{bmatrix} \mathbf{H}_{uu} & \mathbf{H}_{u\phi} \\ \mathbf{H}_{\phi u} & \mathbf{H}_{\phi\phi} \end{bmatrix} \begin{bmatrix} \mathbf{y} \\ \boldsymbol{\eta} \end{bmatrix} = \begin{bmatrix} \mathbf{x}_k \\ \mathbf{0} \end{bmatrix}.$$

The unknown $\mathbf{y}$ is the displacement part of the coupled solution. The unknown $\boldsymbol{\eta}$ is the electric-potential part needed to satisfy the coupled system during this solve.

The block equations are

$$\begin{aligned} \mathbf{H}_{uu}\mathbf{y} + \mathbf{H}_{u\phi}\boldsymbol{\eta} &= \mathbf{x}_k, \\ \mathbf{H}_{\phi u}\mathbf{y} + \mathbf{H}_{\phi\phi}\boldsymbol{\eta} &= \mathbf{0}. \end{aligned}$$

The second equation can be solved for the electric part:

$$\boldsymbol{\eta} = -\mathbf{H}_{\phi\phi}^{-1} \mathbf{H}_{\phi u} \mathbf{y}.$$

Substituting this expression into the first equation removes $\boldsymbol{\eta}$:

$$\left(\mathbf{H}_{uu} - \mathbf{H}_{u\phi} \mathbf{H}_{\phi\phi}^{-1} \mathbf{H}_{\phi u} \right) \mathbf{y} = \mathbf{x}_k.$$

The matrix in parentheses is exactly the condensed mechanical Hessian $\widehat{\mathbf{H}}$. Therefore

$$\widehat{\mathbf{H}} \mathbf{y} = \mathbf{x}_k.$$

Thus

$$\mathbf{y} = \widehat{\mathbf{H}}^{-1} \mathbf{x}_k.$$

This shows why solving the full coupled system gives the same mechanical vector that would be obtained by applying the inverse condensed mechanical operator.

Updating the Eigenvector Direction

After the coupled solve, the code keeps only the displacement part $\mathbf{y}$. This vector is the inverse-iteration update direction. The code then normalizes it:

$$\mathbf{x}_{k+1} = \frac{\mathbf{y}}{\|\mathbf{y}\|}.$$

Therefore one outer iteration is

$$\mathbf{y}_k = \widehat{\mathbf{H}}^{-1} \mathbf{x}_k, \quad \mathbf{x}_{k+1} = \frac{\mathbf{y}_k}{\|\mathbf{y}_k\|}.$$

This is inverse iteration applied to the condensed mechanical Hessian.

Why This Finds the Eigenvector Closest to Zero

If $\widehat{\mathbf{H}}$ has eigenpairs

$$\widehat{\mathbf{H}} \mathbf{q}_i = \lambda_i \mathbf{q}_i,$$

then

$$\widehat{\mathbf{H}}^{-1} \mathbf{q}_i = \frac{1}{\lambda_i} \mathbf{q}_i.$$

To see the amplification more explicitly, write the current iterate as a linear combination of eigenvectors:

$$\mathbf{x}_k = \sum_i c_i^{(k)} \mathbf{q}_i.$$

Here $c_i^{(k)}$ measures how much of eigenvector $\mathbf{q}_i$ is present in the current iterate. Applying the inverse operator gives

$$\mathbf{y}_k = \widehat{\mathbf{H}}^{-1} \mathbf{x}_k = \sum_i c_i^{(k)} \widehat{\mathbf{H}}^{-1} \mathbf{q}_i = \sum_i \frac{c_i^{(k)}}{\lambda_i} \mathbf{q}_i.$$

Thus each eigenvector component is multiplied by the factor

$$\frac{1}{\lambda_i}.$$

This is the central reason inverse iteration works. Components corresponding to smaller $|\lambda_i|$ are multiplied by larger $|1/\lambda_i|$. Suppose λ_1 is the eigenvalue closest to zero, so that

$$|\lambda_1| < |\lambda_j|, \quad j \neq 1.$$

Then after one inverse application, the relative size of the $\mathbf{q}_1$ component compared with the $\mathbf{q}_j$ component changes as

$$\left| \frac{c_1^{(k)}/\lambda_1}{c_j^{(k)}/\lambda_j} \right| = \left| \frac{c_1^{(k)}}{c_j^{(k)}} \right| \left| \frac{\lambda_j}{\lambda_1} \right|.$$

Because $|\lambda_j/\lambda_1| > 1$, the relative contribution of $\mathbf{q}_1$ grows after each inverse iteration. In other words, even if $\mathbf{q}_1$ is only a small part of the initial vector, it is magnified faster than the other eigenvector components. After m ideal inverse applications,

$$\mathbf{x}_{k+m} \text{ is dominated by } \mathbf{q}_1,$$

provided the initial vector has a nonzero component in the $\mathbf{q}_1$ direction.

This gives the main requirement on the initial vector. If

$$\mathbf{x}_0 = \sum_i c_i^{(0)} \mathbf{q}_i,$$

then convergence to $\mathbf{q}_1$ requires

$$c_1^{(0)} \neq 0.$$

If $c_1^{(0)} = 0$ exactly, inverse iteration cannot create that missing component, because each application of $\widehat{\mathbf{H}}^{-1}$ only rescales the coefficients already present:

$$\widehat{\mathbf{H}}^{-1} \mathbf{x}_0 = \sum_i \frac{c_i^{(0)}}{\lambda_i} \mathbf{q}_i.$$

Therefore the $\mathbf{q}_1$ coefficient remains zero for all later iterations. In practice, this exact orthogonality is unlikely if a generic or random-looking initial vector is used. The code initializes $\mathbf{x}_0$ with a deterministic sine-cosine pattern, which is intended to give a nonzero projection onto the critical eigenvector while avoiding the need for random number generation.

The normalization step,

$$\mathbf{x}_{k+1} = \frac{\mathbf{y}_k}{\|\mathbf{y}_k\|}$$

does not change which eigenvector components are relatively larger or smaller. It only prevents the vector norm from growing without bound. Therefore the direction of the iterate converges to the eigenvector whose eigenvalue has the largest reciprocal magnitude, which is the eigenvalue closest to zero in magnitude.

In a stability calculation, this mode is important because loss of stability occurs when a mechanical eigenvalue approaches and crosses zero. The inverse iteration therefore naturally focuses on the critical mode near loss of stability.

4 Eigenvalue Estimate During Iteration

During inverse iteration, the code needs a scalar estimate of the eigenvalue so it can decide whether the outer iteration has converged. At each iteration, the code already has two vectors:

$$\mathbf{x},$$

the current normalized displacement vector, and

$$\mathbf{y},$$

the result of applying the inverse condensed operator to $\mathbf{x}$. That is,

$$\mathbf{y} = \widehat{\mathbf{H}}^{-1} \mathbf{x},$$

where $\mathbf{y}$ is obtained from the full coupled solve described in Section 3.

The code then computes

$$\mu = \mathbf{x}^T \mathbf{y}$$

and stores this value in the variable `inverseEigenvalue`:

$$\text{inverseEigenvalue} = \mathbf{x} \cdot \mathbf{y}.$$

This quantity is called `inverseEigenvalue` because it is an estimate of $1/\lambda$, not λ itself.

To see why, first consider the ideal case where the current vector is exactly an eigenvector:

$$\mathbf{x} = \mathbf{q}, \quad \widehat{\mathbf{H}}\mathbf{q} = \lambda\mathbf{q}, \quad \|\mathbf{q}\| = 1.$$

Applying the inverse condensed operator gives

$$\mathbf{y} = \widehat{\mathbf{H}}^{-1}\mathbf{q} = \frac{1}{\lambda}\mathbf{q}.$$

Then the dot product is

$$\mu = \mathbf{q}^T \left(\frac{1}{\lambda}\mathbf{q} \right) = \frac{1}{\lambda},$$

because $\mathbf{q}^T\mathbf{q} = 1$. Therefore

$$\lambda = \frac{1}{\mu}.$$

In the actual iteration, $\mathbf{x}$ is not exactly equal to the eigenvector until convergence. However, after several inverse-iteration steps, $\mathbf{x}$ becomes close to the critical eigenvector. Then

$$\mathbf{y} = \widehat{\mathbf{H}}^{-1}\mathbf{x} \approx \frac{1}{\lambda}\mathbf{x},$$

so

$$\mathbf{x}^T\mathbf{y} \approx \frac{1}{\lambda}\mathbf{x}^T\mathbf{x}.$$

Since the code normalizes $\mathbf{x}$, we have

$$\mathbf{x}^T\mathbf{x} = 1,$$

and therefore

$$\mathbf{x}^T\mathbf{y} \approx \frac{1}{\lambda}.$$

This gives the iteration-level eigenvalue estimate

$$\lambda \approx \frac{1}{\mathbf{x}^T\mathbf{y}}.$$

In the code, this appears as

$$\text{smallestEigenvalue} = \frac{1}{\text{inverseEigenvalue}}.$$

This estimate is used only to monitor convergence during inverse iteration. The code compares the current estimate with the previous one:

$$|\lambda_k - \lambda_{k-1}|.$$

The iteration stops when this change is small relative to the current eigenvalue scale:

$$|\lambda_k - \lambda_{k-1}| < \text{tolerance} \max(1, |\lambda_k|).$$

After the eigenvector has converged, the code recomputes the eigenvalue using the Rayleigh quotient described in the next section.

5 Final Rayleigh Quotient

After convergence, the function recomputes the eigenvalue using the Rayleigh quotient of the condensed Hessian:

$$\lambda = \mathbf{x}^T \widehat{\mathbf{H}} \mathbf{x}.$$

The code evaluates this without explicitly storing the dense Schur complement:

$$\widehat{\mathbf{H}} \mathbf{x} = \mathbf{H}_{uu} \mathbf{x} - \mathbf{H}_{u\phi} \mathbf{H}_{\phi\phi}^{-1} \mathbf{H}_{\phi u} \mathbf{x}.$$

The important point is that the Rayleigh quotient only needs the vector $\widehat{\mathbf{H}} \mathbf{x}$. It does not need every entry of $\widehat{\mathbf{H}}$. Therefore the code evaluates the product $\widehat{\mathbf{H}} \mathbf{x}$ from right to left.

First, it computes the electric residual produced by the mechanical eigenvector:

$$\mathbf{r} = \mathbf{H}_{\phi u} \mathbf{x}.$$

In the code, this is

$$\text{hpuX} = \mathbf{H}_{\phi u} \mathbf{x}.$$

Second, it solves the electric block system

$$\mathbf{H}_{\phi\phi} \mathbf{w} = \mathbf{r}.$$

Thus

$$\mathbf{w} = \mathbf{H}_{\phi\phi}^{-1} \mathbf{H}_{\phi u} \mathbf{x}.$$

In the code, this is

$$\text{invHppHpuX} = \mathbf{H}_{\phi\phi}^{-1} \mathbf{H}_{\phi u} \mathbf{x}.$$

Third, it forms the condensed mechanical matrix-vector product

$$\mathbf{z} = \mathbf{H}_{uu} \mathbf{x} - \mathbf{H}_{u\phi} \mathbf{w}.$$

Since $\mathbf{w} = \mathbf{H}_{\phi\phi}^{-1} \mathbf{H}_{\phi u} \mathbf{x}$, this is exactly

$$\mathbf{z} = \widehat{\mathbf{H}} \mathbf{x}.$$

In the code, this vector is named `hhatX`:

$$\text{hhatX} = \mathbf{H}_{uu} \mathbf{x} - \mathbf{H}_{u\phi} \text{invHppHpuX}.$$

Finally, the Rayleigh quotient is computed as

$$\lambda = \mathbf{x}^T \mathbf{z} = \mathbf{x}^T \widehat{\mathbf{H}} \mathbf{x}.$$

In the code, this is

$$\text{smallestEigenvalue} = \mathbf{x} \cdot \text{hhatX}.$$

This treatment is different from the inverse iteration step in Section 3. In Section 3, the code applies the inverse condensed operator,

$$\widehat{\mathbf{H}}^{-1} \mathbf{x},$$

by solving the full coupled system with right-hand side $[\mathbf{x}; \mathbf{0}]$. That solve produces the next iterate direction $\mathbf{y} = \widehat{\mathbf{H}}^{-1} \mathbf{x}$. In the final Rayleigh quotient, the code applies the condensed operator itself,

$$\widehat{\mathbf{H}} \mathbf{x},$$

using two block matrix-vector products and one solve with the electric block $\mathbf{H}_{\phi\phi}$. Therefore Section 3 uses the full coupled solve to move the eigenvector iteration forward, while Section 5 uses the Schur-complement formula to evaluate the eigenvalue of the already converged vector.

This final Rayleigh quotient is usually more directly tied to the condensed mechanical operator than the reciprocal estimate used during the iteration.

6 Recovered Full Eigenvector

The displacement eigenvector returned by the function is

$$\delta \mathbf{u} = \mathbf{x}.$$

The associated electric potential eigenvector is recovered from the condensation relation:

$$\delta \phi = -\mathbf{H}_{\phi\phi}^{-1} \mathbf{H}_{\phi u} \mathbf{x}.$$

Therefore the full coupled perturbation is

$$\delta \mathbf{z} = \begin{bmatrix} \delta \mathbf{u} \\ \delta \phi \end{bmatrix} = \begin{bmatrix} \mathbf{x} \\ -\mathbf{H}_{\phi\phi}^{-1} \mathbf{H}_{\phi u} \mathbf{x} \end{bmatrix}.$$

7 Assumptions

The procedure relies on the following assumptions:

- The global tangent matrix is ordered as displacement blocks followed by the electric potential block.
- The electric block $\mathbf{H}_{\phi\phi}$ is nonsingular.
- The condensed mechanical Hessian $\widehat{\mathbf{H}}$ is the correct stability operator for the chosen electrical boundary conditions.
- The eigenvalue of interest is the one closest to zero, so that inverse iteration on $\widehat{\mathbf{H}}^{-1}$ converges to the critical mode.
- The initial vector is nonzero, finite, and has a nonzero projection onto the desired critical eigenvector. If the critical eigenvalue has multiplicity greater than one, the method may converge to a linear combination inside the corresponding eigenspace.

8 Connection to the Implementation

The implementation follows these mathematical steps:

1. Assemble the full tangent matrix $\mathbf{H}$.
2. Split it into $\mathbf{H}_{uu}$, $\mathbf{H}_{u\phi}$, $\mathbf{H}_{\phi u}$, and $\mathbf{H}_{\phi\phi}$.
3. Factorize the full coupled matrix for inverse iteration.
4. Repeatedly solve

$$\mathbf{H} \begin{bmatrix} \mathbf{y} \\ \eta \end{bmatrix} = \begin{bmatrix} \mathbf{x} \\ \mathbf{0} \end{bmatrix}.$$

5. Keep only the displacement part $\mathbf{y}$, normalize it, and update $\mathbf{x}$.
6. Estimate the eigenvalue during iteration as

$$\lambda \approx \frac{1}{\mathbf{x}^T \mathbf{y}}.$$

7. After convergence, compute the final value using

$$\lambda = \mathbf{x}^T \widehat{\mathbf{H}} \mathbf{x}.$$

8. Return $\mathbf{x}$ as the displacement eigenvector and recover the electric part using

$$\delta\phi = -\mathbf{H}_{\phi\phi}^{-1} \mathbf{H}_{\phi u} \mathbf{x}.$$